

Improving Structural Coherence in LLM-Based Website Generation via Schema-Constrained Modeling

Abstract—Large Language Models (LLMs) are increasingly used for automated website generation from natural language conversations. However, directly getting code for website generation from LLMs might suffer from structural inconsistencies, missing fields, and unreliable outputs when inputs (prompts) are noisy or incomplete.

This paper proposes a schema-constrained modeling approach that introduces an intermediate structured representation for improving reliability in conversational website generation systems. The proposed method enforces a predefined JSON schema before rendering HTML, ensuring completeness and consistency.

A comparative evaluation between a baseline direct extraction pipeline and the proposed schema-constrained pipeline is conducted using a custom dataset of 150 multi-turn conversations spanning 20 business categories. Metrics such as Field Completeness Score (FCS), Field Correctness Rate (FCR), and Structural Parsing Success (SPS) are used for evaluation. The evaluation is limited to conversational website generation for small business domains.

Results show that the proposed method significantly improves extraction performance, achieving an average FCS of 0.9513 and FCR of 0.8542 compared to 0.7197 and 0.6811 respectively for the baseline. The proposed approach also achieves a Structural Consistency Score (SCS) of 0.9209 compared with 0.7603 for the baseline, while structural parsing success remains near-perfect. The findings highlight the importance of structured intermediate representations in improving robustness of LLM-based generation systems.

Index Terms—Large Language Models, Structured Information Extraction, Schema-Constrained Generation, Conversational Interfaces, Automated Website Generation, Data-to-UI Mapping, Prompt Engineering, Natural Language Processing

I. INTRODUCTION

Recent advancements in Large Language Models (LLMs) have enabled new applications in automated content generation, including conversational website builders. These systems allow users to describe their business in natural language and automatically generate websites without requiring technical expertise, making website creation more accessible to individuals and small businesses.

One such application is *Chat2Site*, a prototype chatbot-driven business website generation system developed in this work. Chat2Site enables users to interact with a conversational interface that collects business information and transforms it into a structured business website through an automated generation pipeline. Instead of requiring users to manually design webpages or configure website builders, the system generates website content directly from conversational inputs.

Despite these advancements, direct LLM-based website generation still suffers from several limitations, including missing business information such as email addresses, phone numbers, and services, inconsistent output structures, high sensitivity to prompt phrasing, and reduced robustness when conversations are incomplete or ambiguous. These issues frequently produce incomplete or structurally inconsistent websites that require additional manual correction.

Most existing LLM-based website generation systems rely on implicit structural knowledge learned during model training. Consequently, they cannot guarantee the presence of mandatory fields, consistent formatting, or reliable extraction of essential business information before webpage generation.

To address these challenges, this paper proposes a schema-constrained modeling approach in which a predefined JSON schema serves as an intermediate representation between conversational input and HTML generation. By validating the structured representation before rendering, the proposed framework improves the completeness and consistency of the extracted information while reducing structural errors in the generated webpages.

The proposed methodology is evaluated by comparing a conventional direct extraction pipeline with a schema-constrained extraction pipeline using a custom conversational dataset. Experimental results demonstrate that incorporating structured schema validation substantially improves information completeness and extraction correctness, leading to more reliable website generation.

Consequently, the central research question addressed in this paper is:

Can enforcing a predefined schema improve the structural completeness and correctness of LLM-generated outputs for conversational website generation?

II. METHODOLOGY

A. System Overview

Two pipelines are implemented:

Baseline Pipeline:

Conversation → LLM → JSON Output

Proposed Pipeline:

Conversation → LLM + Schema Prompt → Structured JSON → Renderer → HTML

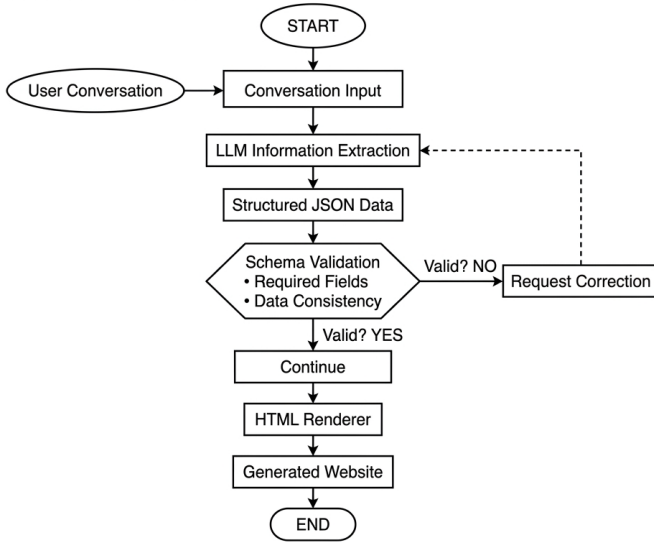


Fig. 1. Overall architecture of the proposed schema-constrained conversational website generation framework.

B. Schema Design

The enforced schema ensures structural consistency:

```

{
  "business_name": "",
  "business_type": "",
  "services": [],
  "city": "",
  "email": "",
  "phone": ""
}
  
```

This schema acts as a constraint that guides the LLM to produce complete outputs.

C. Evaluation Metrics

1) *Field Completeness Score (FCS)*: Measures presence of required fields:

$$FCS = \frac{\text{Number of non-empty fields}}{\text{Total fields}} \quad (1)$$

2) *Field Correctness Rate (FCR)*: Measures correctness of extracted values:

$$FCR = \frac{\text{Correctly extracted fields}}{\text{Total fields}} \quad (2)$$

3) *Structural Parsing Success (SPS)*: Measures validity of JSON:

$$SPS = \frac{\text{Valid JSON outputs}}{\text{Total samples}} \quad (3)$$

III. PIPELINE ARCHITECTURE

The proposed framework follows a sequential pipeline that transforms natural language conversations into structured, schema-compliant websites, as illustrated in Fig. 2. The

pipeline separates information extraction, schema validation, and webpage rendering into independent stages to improve reliability and maintain consistency.

Initially, a Large Language Model (LLM) extracts business information from the conversation and organizes it into a structured JSON representation based on a predefined website schema. The generated JSON is then validated to ensure that all mandatory fields are present and internally consistent.

If missing or invalid information is detected, the LLM regenerates only the affected portions of the JSON instead of repeating the entire extraction process. This validation loop continues until the schema requirements are satisfied.

The validated JSON is finally passed to the HTML rendering module, which populates predefined website templates and generates the corresponding webpages. By introducing schema validation as an intermediate stage, the proposed pipeline minimizes missing information, reduces structural inconsistencies, and improves the robustness of conversational website generation.

Chat2Site Pipeline Architecture

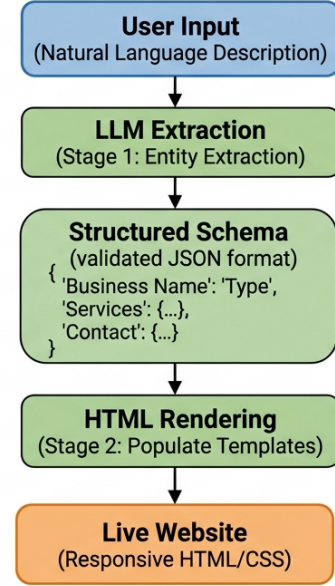


Fig. 2. Proposed Schema-Constrained Pipeline

IV. FORMAL PROBLEM DEFINITION AND MATHEMATICAL FRAMEWORK

This section formalizes the problem of structural reliability in LLM-based website generation and introduces quantitative measures to evaluate structural coherence.

A. Problem Definition

Let C denote a conversational input consisting of a sequence of user utterances:

$$C = \{u_1, u_2, \dots, u_n\} \quad (4)$$

The goal of the system is to generate a structured representation S :

$$S = f(C) \quad (5)$$

where S is defined over a schema $\mathcal{K}$:

$$\mathcal{K} = \{k_1, k_2, \dots, k_m\} \quad (6)$$

Each key k_i corresponds to a required field (e.g., business_name, services, email).

The ground truth structured representation is denoted as S^* .

The objective is to minimize the discrepancy between S and S^* while ensuring structural validity.

B. Definition of Structural Drift

We define *Structural Drift* as the deviation of the predicted structure from the expected schema-compliant structure.

Let:

$$\delta_i = \begin{cases} 1, & \text{if field } k_i \text{ is missing or empty in } S \\ 0, & \text{otherwise} \end{cases} \quad (7)$$

Then, the **Structural Drift Score (SDS)** is defined as:

$$SDS = \frac{1}{m} \sum_{i=1}^m \delta_i \quad (8)$$

This measures the proportion of missing or incomplete fields.

Additionally, we define **Semantic Drift** to capture incorrect field values:

$$\epsilon_i = \begin{cases} 1, & \text{if } S[k_i] \neq S^*[k_i] \\ 0, & \text{otherwise} \end{cases} \quad (9)$$

$$Semantic\ Drift = \frac{1}{m} \sum_{i=1}^m \epsilon_i \quad (10)$$

Together, these quantify the "reliability gap" between generated and expected outputs.

C. Structural Consistency Score (SCS)

To combine completeness, correctness, and structural validity into a single metric, we define the Structural Consistency Score (SCS):

$$SCS = \alpha \cdot FCS + \beta \cdot FCR + \gamma \cdot SPS \quad (11)$$

subject to:

$$\alpha + \beta + \gamma = 1, \quad \alpha, \beta, \gamma \in [0, 1] \quad (12)$$

where:

- FCS = Field Completeness Score
- FCR = Field Correctness Rate
- SPS = Structural Parsing Success

D. Interpretation of Weights

The weights α , β , and γ determine the relative importance of completeness, correctness, and structural validity.

1) *Default Configuration*: In general-purpose website generation:

$$\alpha = 0.4, \quad \beta = 0.4, \quad \gamma = 0.2 \quad (13)$$

Rationale:

- Completeness and correctness are equally critical
- Structural validity is necessary but less dominant once parsing succeeds

TABLE I
PERFORMANCE COMPARISON WITH STRUCTURAL CONSISTENCY SCORE (SCS)

Model	FCS	FCR	SPS	SCS	Improvement
Baseline	0.7197	0.6811	1.0000	0.7603	–
Structured	0.9513	0.8542	0.9935	0.9209	+21.1%

E. Optimization Objective

The system aims to maximize SCS:

$$\max_f SCS(f(C)) \quad (14)$$

subject to:

- Schema compliance constraints
- Valid JSON output
- Minimal hallucination

F. Key Insight

The proposed schema-constrained approach reduces both Structural Drift and Semantic Drift by enforcing explicit constraints during generation. This transforms the problem from unconstrained text generation into a structured prediction task, leading to improved reliability and consistency.

V. SYSTEM ARCHITECTURE

This section presents a detailed description of the Chat2Site pipeline, focusing on the transformation from unstructured conversational input to structured website output. The system is designed as a multi-stage pipeline consisting of preprocessing, structured extraction, validation, and template-based rendering.

A. Pipeline Overview

The proposed system follows a modular architecture:

Conversation $\rightarrow$ Preprocessing $\rightarrow$ LLM Extraction $\rightarrow$ Validation Layer $\rightarrow$ Template Mapping $\rightarrow$ HTML Output

Each stage is designed to reduce ambiguity and enforce structural consistency.

B. Preprocessing and Prompt Engineering

The input to the system consists of multi-turn conversational text, which is inherently noisy and unstructured. The preprocessing step standardizes the input before passing it to the LLM.

1) *Conversation Normalization*: The system concatenates multi-turn conversations into a unified format:

User: I run a cake shop called Kelvin
Cakes.
User: We offer birthday cakes and
cupcakes.
User: Contact: hello@kelvin.com

This ensures consistency in LLM input and reduces ambiguity caused by fragmented messages.

2) *Prompt Engineering Strategy*: A carefully designed system prompt is used to enforce structured extraction. The prompt explicitly defines:

- Required fields
- Output format constraints
- Strict JSON compliance

System Prompt Used:

You are an information extraction system.

Extract structured data from
the conversation.

STRICT RULES:

- Return ONLY JSON
- Do NOT add explanations
- Do NOT add text before or after JSON
- Ensure valid JSON format
(no trailing commas)

Format:

```
{  
  "business_name": "",  
  "business_type": "",  
  "services": [],  
  "city": "",  
  "email": "",  
  "phone": ""  
}
```

3) *Design Rationale*: Unlike baseline approaches that rely on implicit structure, this prompt enforces:

- Deterministic output format
- Reduced hallucination
- Improved field completeness

This step is critical in ensuring that downstream components receive consistent input.

C. Validation Layer

The validation layer acts as a safeguard between LLM output and rendering. Its purpose is to ensure correctness, completeness, and usability of extracted data.

1) *Validation Checks*: The system performs the following checks:

- JSON Parsing Validation (syntactic correctness)
- Field Presence Validation
- Data Type Validation (e.g., list vs string)
- Pattern Validation (email, phone format)

2) *Correction and Recovery Mechanism*: If validation fails, the system attempts recovery using:

- Automatic JSON repair (for minor syntax issues)
- Re-prompting the LLM with stricter instructions

3) *Clarification Requests*: In cases where critical fields are missing (e.g., business name or contact information), the system generates clarification prompts.

Example:

"Could you please provide your
business email or phone number?"

This introduces an interactive loop:

LLM Output → Validation Failure → User Clarification →
Re-extraction

4) *Sequence Flow*:

```
User Input → LLM → JSON Output  
      ↓  
      Validation Layer  
      ↓  
      [Valid] → Renderer  
      [Invalid] → Repair / Clarification → LLM
```

This ensures robustness in real-world scenarios where user input may be incomplete.

D. Template Mapping Engine

The template mapping engine converts structured JSON into a fully functional HTML website. This component bridges the gap between structured data and visual representation.

1) *Mapping Strategy*: Each JSON field is mapped to predefined HTML components:

- **business_name** → Header / Hero Section
- **business_type** → Tagline or subtitle
- **services** → Services section (cards/list)
- **city** → Location display
- **email, phone** → Contact section

2) *Example Mapping*:

```
<h1>{{business_name}}</h1>  
<p>{{business_type}}</p>  
  
<ul>  
  {% for service in services %}  
    <li>{{service}}</li>  
  {% endfor %}  
</ul>  
  
<p>Email: {{email}}</p>  
<p>Phone: {{phone}}</p>
```

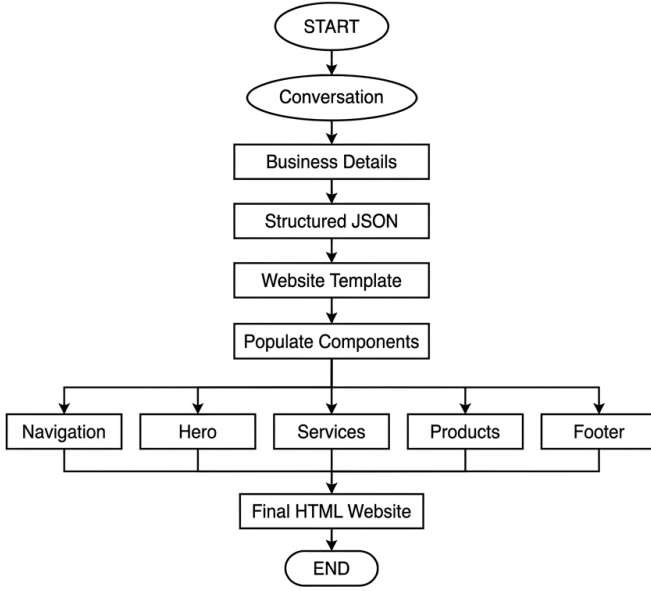


Fig. 3. Workflow of template-based website generation from structured JSON data.

3) *Design Principles*: The mapping engine follows:

- Separation of content and presentation
- Reusability of templates
- Minimal dependency on LLM formatting

4) *Why This Matters*: In baseline approaches, LLM directly generates HTML, leading to:

- Inconsistent layouts
- Missing sections
- Broken formatting

By contrast, the structured approach ensures:

- Guaranteed section presence
- Clean separation of logic
- Consistent UI generation

E. End-to-End Flow

The complete pipeline ensures:

- Robust handling of noisy inputs
- Improved extraction accuracy
- Reliable website generation

The introduction of schema constraints and validation transforms the system from a generative pipeline into a controlled, semi-deterministic architecture.

VI. EXPERIMENTAL SETUP

A. Dataset

A custom conversational dataset was developed to evaluate the proposed schema-constrained website generation framework. The dataset consists of 150 multi-turn conversations spanning 20 business categories, including restaurants, bakeries, beauty salons, gyms, dental clinics, coaching centers, bookstores, travel agencies, pet shops, grocery stores, and other small businesses. Each conversation contains business

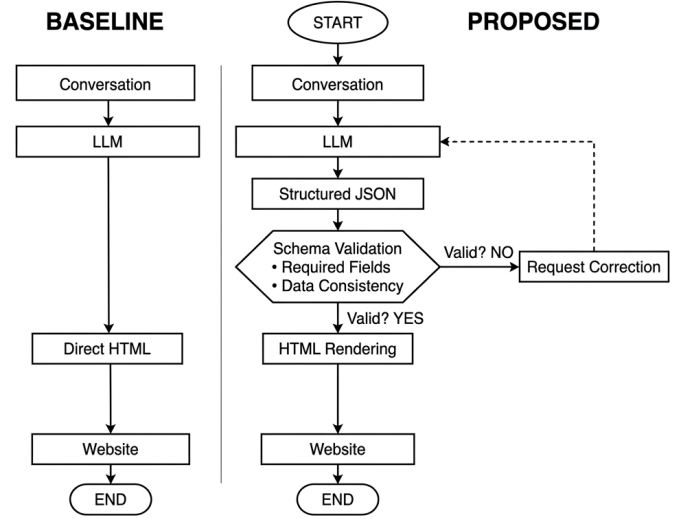


Fig. 4. Comparison between the baseline direct generation pipeline and the proposed schema-constrained pipeline.

details such as the business name, category, services, city, email address, and phone number.

For every conversation, a manually prepared ground-truth JSON annotation containing 6 mandatory fields was created as the reference for evaluating extraction quality.

The framework employs a predefined JSON schema as an intermediate representation before HTML generation. Besides the core business information, the schema includes website-specific components such as branding, navigation menus, hero sections, products, categories, promotional offers, and footer details, ensuring structural consistency.

To evaluate extraction performance, 150 prediction outputs were generated using both a direct extraction baseline and the proposed schema-constrained approach. Both prediction sets were compared against the same ground-truth annotations using the proposed evaluation metrics.

B. Implementation

The experimental evaluation compares two extraction approaches:

- **Direct Extraction Baseline**: Business information is extracted directly from conversational text without specialized prompt engineering.
- **Prompt-Based Baseline**: Structured extraction is performed using an improved prompt designed to increase field coverage before schema validation and website rendering.

For each conversation, the predicted JSON output is compared with the manually prepared ground-truth annotation. Performance is evaluated using the proposed metrics: Field Completeness Score (FCS), Field Correctness Rate (FCR), Structural Parsing Success (SPS), and Structural Consistency Score (SCS).

C. Experimental Environment

All experiments were conducted using identical conversational inputs and evaluation criteria to ensure a fair comparison between extraction strategies. The extracted JSON outputs were validated against the predefined website schema before being passed to the rendering module. By introducing schema validation as an intermediate step, the proposed framework reduces structural inconsistencies and improves the reliability of automatically generated webpages.

VII. RESULTS

A. Quantitative Evaluation

TABLE II
PERFORMANCE COMPARISON

Model	FCS	FCR	SPS
Baseline	0.7197 ± 0.0065	0.6811 ± 0.0035	1.0000
Structured	0.9513 ± 0.0053	0.8542 ± 0.0022	0.9935 ± 0.0053

B. Analysis

The structured approach significantly improves field completeness, extraction accuracy, and output consistency. FCS increased from 0.7197 to 0.9513, while FCR improved from 0.6811 to 0.8542. Although SPS decreased slightly from 1.0000 to 0.9935, the reduction is negligible and results from stricter schema validation. An end-to-end validation using approximately ten representative conversational scenarios further showed that the improved structured extraction produced more complete and consistent HTML webpages than the direct extraction pipeline.

VIII. DISCUSSION

The results indicate that schema constraints improve the reliability of conversational website generation by reducing missing information from clumsy conversations in which baseline fails to extract the required section before webpage rendering. Manual inspection of the generated HTML outputs confirmed that improvements in the intermediate JSON representation translated into more complete and structurally consistent webpages. The modular separation of extraction and rendering also enables reusable website templates across different business domains.

IX. LIMITATIONS

- Limited evaluation dataset focused on business websites.
- HTML validation was performed on approximately ten representative conversations through manual inspection.
- Rendering quality depends on the underlying website templates.

X. FUTURE WORK

Future work includes:

- Evaluating full website quality using UX metrics
- Expanding schema to include design attributes
- Handling multilingual conversational inputs
- Integration with production-grade website builders

XI. CONCLUSION

This paper presents a schema-constrained modeling approach for improving structural coherence in LLM-based website generation systems. The proposed method significantly improves completeness and correctness of extracted data, making it more robust for real-world applications.

REFERENCES

- [1] G. Poesia, O. Polozov, V. Le *et al.*, “Synchromesh: Reliable Code Generation from Pre-trained Language Models,” *arXiv preprint arXiv:2201.11227*, Jan. 2022.
- [2] L. Zheng, L. Yin, Z. Xie *et al.*, “Efficient Execution of Structured Language Model Programs,” *arXiv preprint arXiv:2312.07104*, Dec. 2023.
- [3] H.-V. Tran, T.-T. Dang, N. D. Anh *et al.*, “Automated Website Generation from Heterogeneous Artifacts,” *ResearchGate*, May 2026.
- [4] B. Rozière, J. Gehring, F. Gloeckle *et al.*, “Code Llama: Open Foundation Models for Code,” *arXiv preprint arXiv:2308.12950*, Aug. 2023.
- [5] OpenAI, “GPT-4 Technical Report,” *arXiv preprint arXiv:2303.08774*, Mar. 2023.
- [6] H. Touvron, L. Martin, K. Stone *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” *arXiv preprint arXiv:2307.09288*, Jul. 2023.
- [7] Y. Li, S. Choi, J. Chung *et al.*, “StarCoder: May the Source Be with You!,” *arXiv preprint arXiv:2305.06161*, May 2023.
- [8] L. Ouyang, J. Wu, X. Jiang *et al.*, “Training Language Models to Follow Instructions with Human Feedback,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 27730–27744, 2022.
- [9] S. Bubeck, V. Chandrasekaran, R. Eldan *et al.*, “Sparks of Artificial General Intelligence: Early Experiments with GPT-4,” *arXiv preprint arXiv:2303.12712*, Mar. 2023.
- [10] J. Wei, X. Wang, D. Schuurmans *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 24824–24837, 2022.